

UNIVERSITI TEKNOLOGI MALAYSIA

SECP3623 - DATABASE PROGRAMMING 2025/2026 - SEM 1

Lab Assignment 1 (5%) - Indexes

Name	:	ELIJAH SHE YU SHENG
Matric Id	:	A23CS0073

Name	:	CHUA JIA LIN
Matric Id	:	A23CS0069

Name	:	CHERYL CHEONG KAH VOON
Matric Id	:	A23CS0060

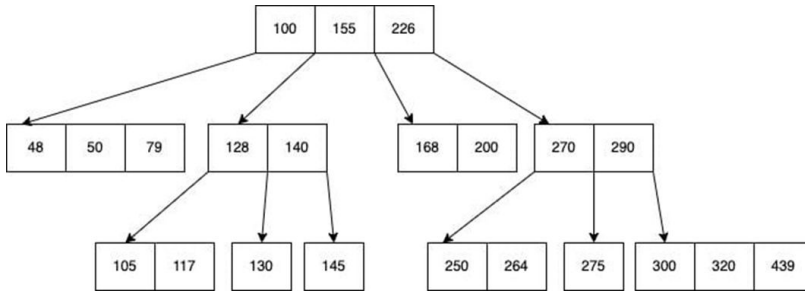
UNIVERSITI TEKNOLOGI MALAYSIA

B-TREE INDEXES

SECTION 2 - GROUP 4



What is B-Tree index?



1. B-tree stores **sorted data** for fast search, insert, and delete.
2. Used in systems that handle **large data blocks**.
3. Each node can have **multiple keys** (not just two).
4. Keys in a node are stored in **ascending order**.
5. **Left child < key < right child** rule applies.
6. A node with **n keys** can have up to **n + 1 children**.

Advantages, Disadvantages & Comparison

- | | |
|--|---|
| <ol style="list-style-type: none">1. Fast searching, insertion, and deletion : operations are efficient ($O \log n$).2. Sorted data access : supports both random and sequential access.3. Balanced structure : keeps data evenly distributed, avoiding long branches. | <ol style="list-style-type: none">1. Slower for small datasets : overhead may not be worth it.2. More complex to maintain than simple indexes.3. Higher memory and storage use : needs extra space for tree structure. |
|--|---|
-
1. **B-tree vs Bitmap**: B-tree is better for frequent updates; Bitmap is better for columns with few distinct values.
 2. **B-tree vs Text/Full-text**: B-tree is for exact or range lookups; Full-text is for word or phrase searches.
 3. **B-tree vs Multicolumn**: B-tree handles single-column well; Multicolumn optimizes combined-column queries.
 4. **B-tree vs Unique**: B-tree focuses on speed; Unique adds value uniqueness enforcement.

Insert 15-20 Rows of Employee Data:

```
INSERT INTO employee(emp_id, emp_name, department, salary,
join_date) VALUES
```

```
(1, 'Alice Tan', 'HR', 4200.00, '2021-03-10'),
(2, 'Ben Lee', 'Finance', 6500.00, '2019-11-21'),
(3, 'Chong Wei', 'IT', 7200.00, '2020-07-15'),
(4, 'David Ong', 'Marketing', 2900.00, '2022-02-18'),
(5, 'Emily Lim', 'Finance', 4800.00, '2021-06-01'),
(6, 'Farah Noor', 'HR', 3000.00, '2019-09-05'),
(7, 'Gavin Tan', 'IT', 8900.00, '2018-01-12'),
(8, 'Hui Ying', 'Admin', 2800.00, '2020-10-25'),
(9, 'Isaac Chong', 'Marketing', 5600.00, '2023-04-19'),
(10, 'Jason Wong', 'Finance', 7000.00, '2022-12-08'),
(11, 'Kelly Ng', 'IT', 5200.00, '2021-05-23'),
(12, 'Lydia Teo', 'Admin', 4600.00, '2020-08-30'),
(13, 'Marcus Lim', 'Finance', 2900.00, '2019-04-10'),
(14, 'Natalie Lee', 'Marketing', 3900.00, '2022-07-14'),
(15, 'Oscar Tan', 'IT', 5800.00, '2020-03-11');
```

emp_id	emp_name	department	salary	join_date
1	Alice Tan	HR	4200.00	2021-03-10
2	Ben Lee	Finance	6500.00	2019-11-21
3	Chong Wei	IT	7200.00	2020-07-15
4	David Ong	Marketing	2900.00	2022-02-18
5	Emily Lim	Finance	4800.00	2021-06-01
6	Farah Noor	HR	3000.00	2019-09-05
7	Gavin Tan	IT	8900.00	2018-01-12
8	Hui Ying	Admin	2800.00	2020-10-25
9	Isaac Chong	Marketing	5600.00	2023-04-19
10	Jason Wong	Finance	7000.00	2022-12-08
11	Kelly Ng	IT	5200.00	2021-05-23
12	Lydia Teo	Admin	4600.00	2020-08-30
13	Marcus Lim	Finance	2900.00	2019-04-10
14	Natalie Lee	Marketing	3900.00	2022-07-14
15	Oscar Tan	IT	5800.00	2020-03-11

Add a B-Tree index on salary:

```
CREATE INDEX idx_salary ON employee(salary);
```

Execute and record query time:

```
SELECT * FROM employee WHERE  
salary BETWEEN 3000 AND 7000;
```

Query time: 0.000 sec

emp_id	emp_name	department	salary	join_date
6	Farah Noor	HR	3000.00	2019-09-05
14	Natalie Lee	Marketing	3900.00	2022-07-14
1	Alice Tan	HR	4200.00	2021-03-10
12	Lydia Teo	Admin	4600.00	2020-08-30
5	Emily Lim	Finance	4800.00	2021-06-01
11	Kelly Ng	IT	5200.00	2021-05-23
9	Isaac Chong	Marketing	5600.00	2023-04-19
15	Oscar Tan	IT	5800.00	2020-03-11
2	Ben Lee	Finance	6500.00	2019-11-21
10	Jason Wong	Finance	7000.00	2022-12-08



55 13:59:26 SELECT * FROM employee WHERE salary BETWEEN ... 10 row(s) returned

0.000 sec / 0.000 sec

Add an index on join_date and compare range query performance

```
CREATE INDEX idx_join_date ON employee(join_date);
```

Execute and record query time:

```
SELECT *  
FROM employee  
WHERE join_date BETWEEN '2022-01-01'  
AND '2024-12-31';
```

Condition	Query Time
Without index	0.000 sec
With index	0.000 sec

✓	107 00:41:01 SELECT * FROM employee WHERE join_date BETWEEN '2020-01-01' AND '2022-12-31' LIMIT 0, 1000	10 row(s) returned	0.000 sec / 0.000 sec
✓	109 00:42:24 CREATE INDEX idx_join_date ON employee(join_date)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.063 sec
✓	110 00:42:26 SELECT * FROM employee WHERE join_date BETWEEN '2020-01-01' AND '2022-12-31' LIMIT 0, 1000	10 row(s) returned	0.000 sec / 0.000 sec

Use EXPLAIN to confirm index usage

EXPLAIN SELECT *

FROM employee

WHERE join_date BETWEEN '2022-01-01' AND '2024-12-31';

Execute and record query time:

Without Index:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	employee	NULL	ALL	NULL	NULL	NULL	NULL	15	11.11	Using where

With Index:

	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
▶	1	SIMPLE	employee	NULL	range	idx_join_date	idx_join_date	4	NULL	4	100.00	Using index condition

Drop Index & Re-run Query

```
DROP INDEX idx_join_date ON employee;
```

Re-run range query

```
SELECT *  
FROM employee  
WHERE join_date BETWEEN '2022-01-01' AND '2024-12-31';
```

Query time: 0.000 sec

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	employee	NULL	ALL	NULL	NULL	NULL	NULL	15	11.11	Using where

Discuss why B-Tree is ideal for range & sorting

B-Tree indexes store values in sorted hierarchical order.

When performing range queries, MySQL can quickly navigate to the start of the range and then read sequentially, avoiding full scans.

Because the index is already sorted, sorting operations (e.g., ORDER BY) are faster as no extra sort step is needed.

This makes B-Tree indexes ideal for range filtering and sorting workloads.

THANK YOU



univteknologimalaysia



utm.my



utmoofficial